

A Modern Ecommerce System Design

Bounded contexts, aggregates, commands, and context relationships (sequence & architecture views)

The system is decomposed into eight bounded contexts, grouped under four teams (Catalog & Discovery, Checkout, Supply, Customer Comms, see Section 4). Checkout is orchestrated by a process manager so that stock reservation, payment authorization, and capture stay strictly ordered. Everything downstream (fulfillment, notifications, search projections) is choreographed through domain events. The whole system still ships as a single deployable process, which keeps cross-team coordination and time-to-ship low.

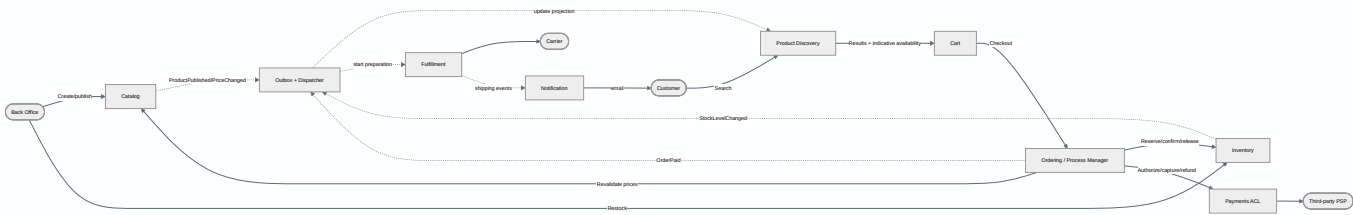
1 Bounded Contexts

Table 1: Bounded contexts and their responsibilities

Context	Responsible for	Not responsible for
Catalog	Product master data, descriptions, media, list prices, publishing	Available quantities
Inventory	Stock levels per product, reservations, restocking, non-overselling invariant	Prices, orders, payments, shipping
Cart	Customer/guest cart, cart lines, and price snapshots	Guaranteeing availability at checkout
Ordering / Checkout	Orders, state machine, checkout orchestration, and compensations	Payment details or the physical handling of stock and shipping
Payments	Abstraction over the payment service, authorization, capture, void/refund, and reconciliation	Stock and availability
Fulfillment	Preparation and shipping of paid orders, tracking	Payments, pricing, and commercial availability
Product Discovery	Full-text index on product names and descriptions, filters, indicative availability	Any domain write operation
Notification	Transactional emails and order/shipping status notifications	Physical order preparation or shipping

Figure 1: Context relationships and event flow

Solid = synchronous call. Dashed = asynchronous domain event via outbox.



2 Aggregates, Entities, and Commands

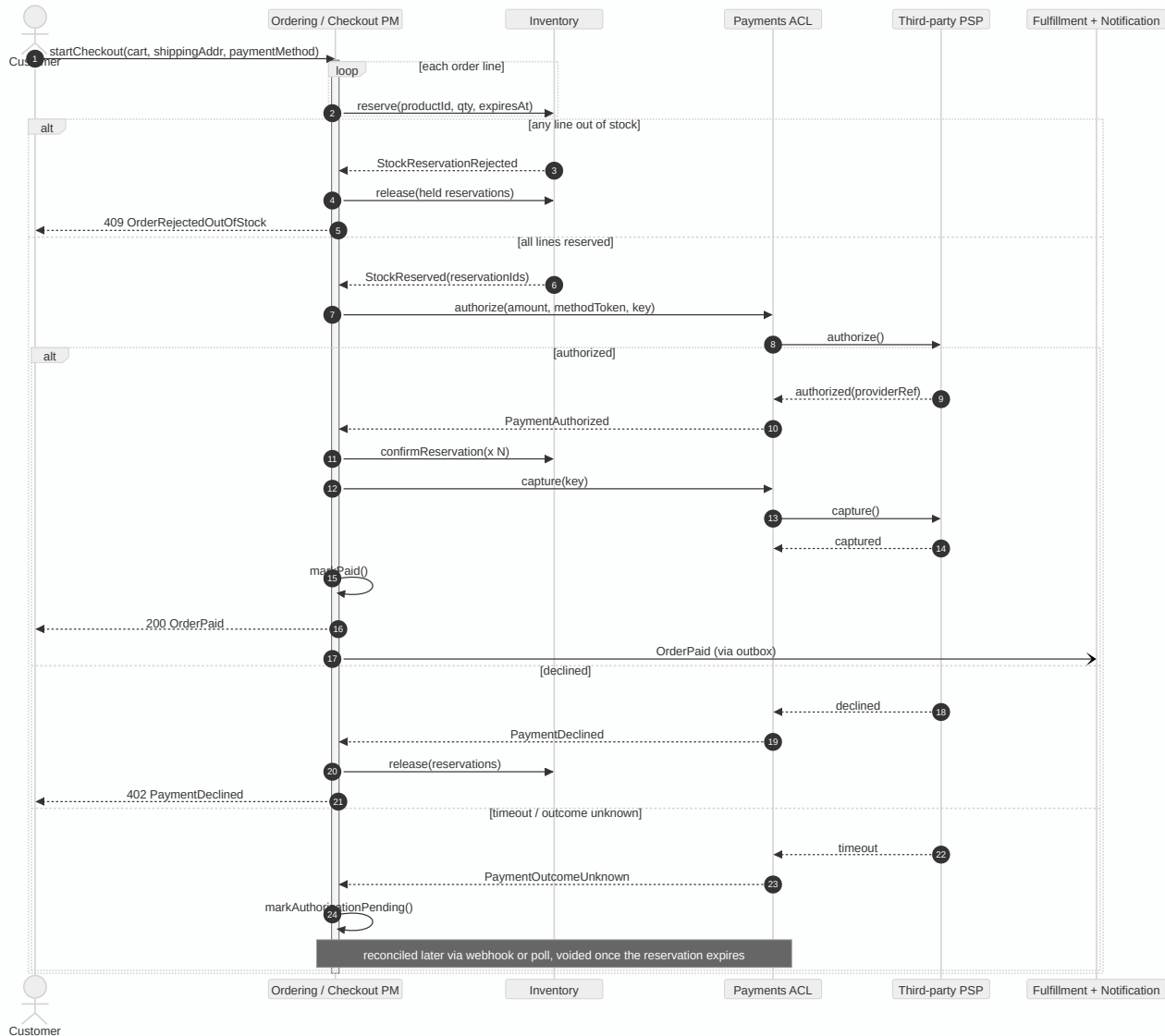
Table 2: Aggregate roots, key commands and invariants

Aggregate	Commands → emitted events	Key invariant
Product Catalog	create, attachMedia, updateDetails, changePrice, publish, discontinue → Product{Created,DetailsUpdated,Published,Discontinued}, PriceChanged	A published product has a name, description, price, and at least 1 image
StockItem Inventory	receiveStock, reserve, confirmReservation, releaseReservation, reverseConfirmation, adjustStock → Stock{Received,Reserved,Confirmed,Released,ConfirmationReversed} + StockLevelChanged	available = onHand - reserved ≥ 0
Cart Cart	addItem, changeQuantity, removeItem, mergeIntoCustomerCart → CartUpdated, CartMerged	Positive quantities, one line per product, not expired or already in checkout
Order Ordering	startCheckout, confirmAllStockReserved, markPaymentAuthorized, confirmStock, markPaid, rejectOutOfStock, cancel → CheckoutStarted ... OrderPaid / OrderRejectedOutOfStock / OrderCancelled	No payment without an active reservation per line. Paid requires confirmed stock and captured payment. Totals frozen once authorization starts
Payment Payments	authorize, capture, voidAuthorization, refund → PaymentAuthorized/Declined/OutcomeUnknown, Captured/CaptureFailed, AuthorizationVoided, Refunded	Retries never duplicate a charge (idempotency key). Active captures ≤ order total
Shipment Fulfillment	prepare, markShipped → ShipmentPrepared, OrderShipped	Created only for a paid order. Cannot ship without tracking info

3 Checkout Sequence: Happy Path, Out-of-Stock, Payment Timeout

Figure 2: Orchestrated checkout (one flow, three outcomes)

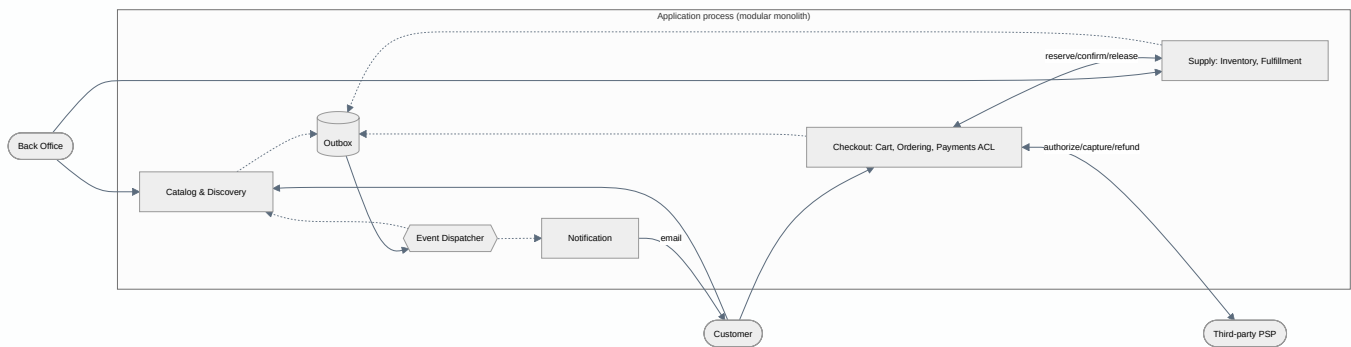
Reservation always precedes authorization, and authorization always precedes capture. Out-of-stock and payment-timeout are modeled as alternative branches of the same interaction, not separate flows.



4 Physical Architecture

Figure 3: Modular monolith, one deployable process

One module per team, with module-owned schemas, and a durable outbox/dispatcher instead of an in-memory bus, so the same code can later publish to an external broker.



Team ownership: **Catalog & Discovery** owns Catalog and Product Discovery. **Checkout** owns Cart, Ordering, and Payments. **Supply** owns Inventory and Fulfillment. **Customer Comms** owns Notification. Modules share no aggregates. The release train is shared, but internals are not.

Why a monolith and not microservices: with two to four teams, starting distributed would add operational cost, distributed observability, and contract management before those costs are justified. Product Discovery, Payments, and Notification are the natural first candidates for extraction, since their load, compliance, and reliability profiles differ most from the rest. Extraction still takes work: clear boundaries and contracts reduce coupling, they do not make a transport change free.

5 Edge Cases and Requirement Coverage

- **Partial reservation failure:** already-held reservations are released before responding. There is no partially-valid multi-product hold. Reservation TTL is the safety net if compensation itself fails to run.
- **Ambiguous payment outcome:** a timeout is not a decline. Webhook and polling reconcile via `ProviderReference / IdempotencyKey`. A late authorization is voided, never captured, if the reservation already expired.
- **Capture/confirmation failure:** authorization is voided and confirmed reservations are compensated via `reverseConfirmation`, both idempotent and audit-logged.
- **Price drift:** prices are revalidated between add-to-cart and checkout. Totals become immutable once authorization starts.

Table 3: Requirement coverage

Requirement	Mechanism
Search by name/description, add to cart if available	Search projection (Fig. 1), best-effort <code>inStock</code> check, authoritative reservation at checkout (Fig. 2)
Back office: product + image/description/qty, restock	<code>Product</code> and <code>StockItem.receiveStock()</code> (Table 2)
Checkout with address + payment method, guest allowed	<code>Order.Buyer</code> : required email, optional <code>customerId</code>
Payments via third party, stock kept independent of payments	<code>Payments</code> ACL never reads stock, <code>Inventory</code> never calls the PSP
Out of stock blocks payment and is shown in the UI	Payment is only ever called after all reservations succeed (Fig. 2, branch 1)
Order preparation, shipping, and notifications	<code>Fulfillment</code> + <code>Notification</code> , event-driven off <code>OrderPaid</code> (Fig. 1)

Impact. The design protects one thing above all: no customer is charged before stock is reserved and confirmed, and no unit is sold twice.

Quality. Scalability: storefront reads are served by a projection that never touches aggregates with invariants, so write contention stays isolated to a single product. Resilience: outbox and inbox prevent dual writes, idempotency and reconciliation make retries and webhooks safe, and TTLs plus compensations protect stock. Responsiveness: projections, notifications, and fulfillment run asynchronously, and the process manager keeps checkout progress observable even when the PSP responds slowly. Accepted costs: stock stays held temporarily for abandoned checkouts, storefront availability can be briefly stale (corrected at checkout), and the process manager plus compensations add complexity to avoid distributed transactions. Monitoring must cover expired reservations, stuck processes, unknown PSP outcomes, and outbox backlog.

Effort. Boundaries align with bounded contexts and no aggregate is shared, so the modular monolith is achievable by a small team while keeping contracts explicit. Modules should only be extracted into services when load, compliance, or team autonomy actually require it.